

www.lumay.ai



LuMay AI



8 ML Algorithms You Should Always Know



#Reshmawithai





Linear Regression

Best for: Predicting continuous values (prices, temperatures, sales)

How it works

- 1 Find the best-fit line through data points
- 2 Minimize the distance between line and points
- 3 Calculate slope (weight) and intercept (bias)
- 4 Use equation: $y = mx + b$ to predict

✓ Pros

Fast, interpretable, simple

✗ Cons

Only linear relationships

⚙️ Key param: **fit_intercept** (include bias term)

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

FOLLOW RESHMA



AND REPOST FOR OTHERS

SWIPE



Logistic Regression

Best for: Binary classification (spam/not spam, yes/no)

How it works

- 1 Apply sigmoid function to linear combination
- 2 Output probability between 0 and 1
- 3 Use threshold (usually 0.5) to classify
- 4 Optimize with log-loss function

✓ Pros

Probability output, works well for linearly separable data

✗ Cons

Assumes linear decision boundary

⚙️ Key param: **C** (inverse regularization strength)

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(C=1.0)
model.fit(X_train, y_train)
proba = model.predict_proba(X_test)
```





Decision Trees

Best for: Interpretable models, handling non-linear data

How it works

- 1 Split data by feature that best separates classes
- 2 Use Gini impurity or entropy to measure splits
- 3 Recursively split until stopping criteria
- 4 Leaf nodes give final prediction

✓ Pros

Easy to visualize, no scaling needed

✗ Cons

Overfits easily, unstable

⚙️ Key param: **max_depth** (tree depth limit)

```
from sklearn.tree import DecisionTreeClassifier model =  
DecisionTreeClassifier(max_depth=5) model.fit(X_train, y_train)  
predictions = model.predict(X_test)
```





Random Forest

Best for: High accuracy, reducing overfitting from decision trees

How it works

- 1 Create many decision trees on random data samples
- 2 Each tree uses random feature subset at splits
- 3 All trees vote on final prediction (bagging)
- 4 Majority vote or average determines output

✓ Pros

Robust, handles missing data, feature importance

✗ Cons

Slow, black-box model

⚙️ Key param: **n_estimators** (number of trees)

```
from sklearn.ensemble import RandomForestClassifier model =  
RandomForestClassifier(n_estimators=100) model.fit(X_train, y_train)  
predictions = model.predict(X_test)
```





Support Vector Machine

Best for: High-dimensional data, clear margin separation

How it works

- 1 Find hyperplane that maximizes margin between classes
- 2 Support vectors are closest points to boundary
- 3 Use kernel trick for non-linear boundaries
- 4 Classify based on which side of plane point falls

✓ Pros

Effective in high dimensions, memory efficient

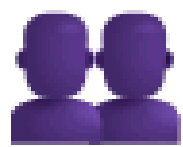
✗ Cons

Slow for large datasets, sensitive to scaling

⚙️ Key param: **kernel** (linear, rbf, poly)

```
from sklearn.svm import SVC
model = SVC(kernel='rbf', C=1.0)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```





K-Nearest Neighbors

Best for: Small datasets, multi-class problems, recommendation systems

How it works

- 1 Store all training data (lazy learning)
- 2 Find K closest neighbors to test point
- 3 Use distance metric (Euclidean, Manhattan)
- 4 Majority vote of K neighbors determines class

✓ Pros

Simple, no training phase, adapts to new data

✗ Cons

Slow predictions, memory intensive

⚙️ Key param: **n_neighbors** (K value)

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```



Naive Bayes

Best for: Text classification (spam detection, sentiment analysis)

How it works

- 1 Apply Bayes' theorem with independence assumption
- 2 Calculate probability of each class given features
- 3 Assume features are independent (naive part)
- 4 Choose class with highest probability

✓ Pros

Fast, works well with small data,
handles high dimensions

✗ Cons

Independence assumption rarely true

⚙️ Key param: **alpha** (smoothing parameter)

```
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```





Neural Networks

Best for: Complex patterns, images, sequences, unstructured data

How it works

- 1 Pass input through layers of neurons (nodes)
- 2 Each neuron applies weights + activation function
- 3 Backpropagation adjusts weights to minimize error
- 4 Repeat for many epochs until convergence

✓ Pros

Handles complex patterns, state-of-the-art accuracy

✗ Cons

Needs lots of data, black-box, slow

⚙️ Key param: **learning_rate** (step size for updates)

```
from sklearn.neural_network import MLPClassifier
model = MLPClassifier(hidden_layer_sizes=(100,50))
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```



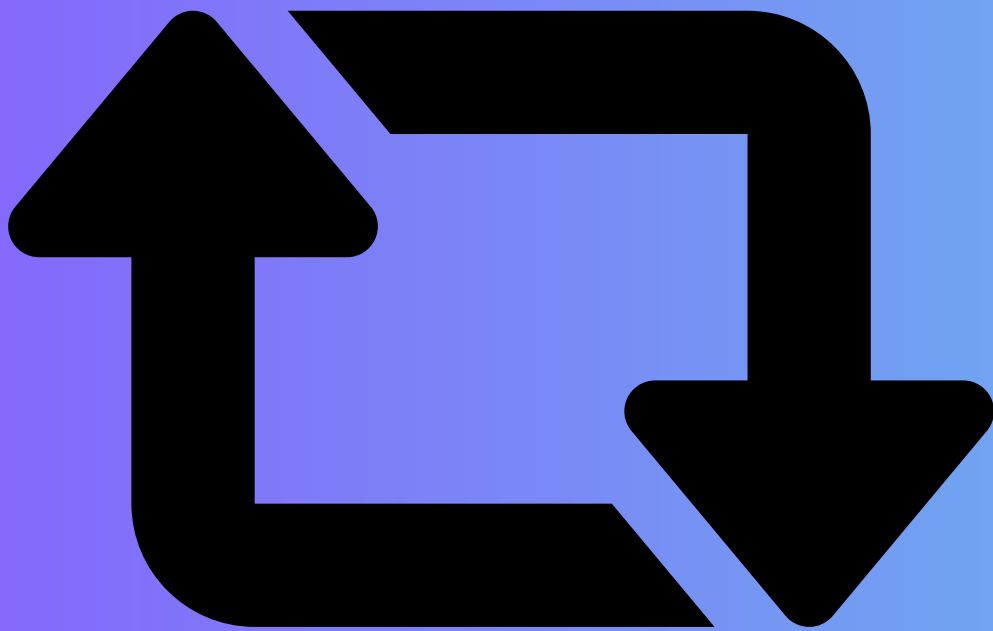


LuMay AI



Repost

If You Found It Helpful



www.lumay.ai



#Reshmawithai

